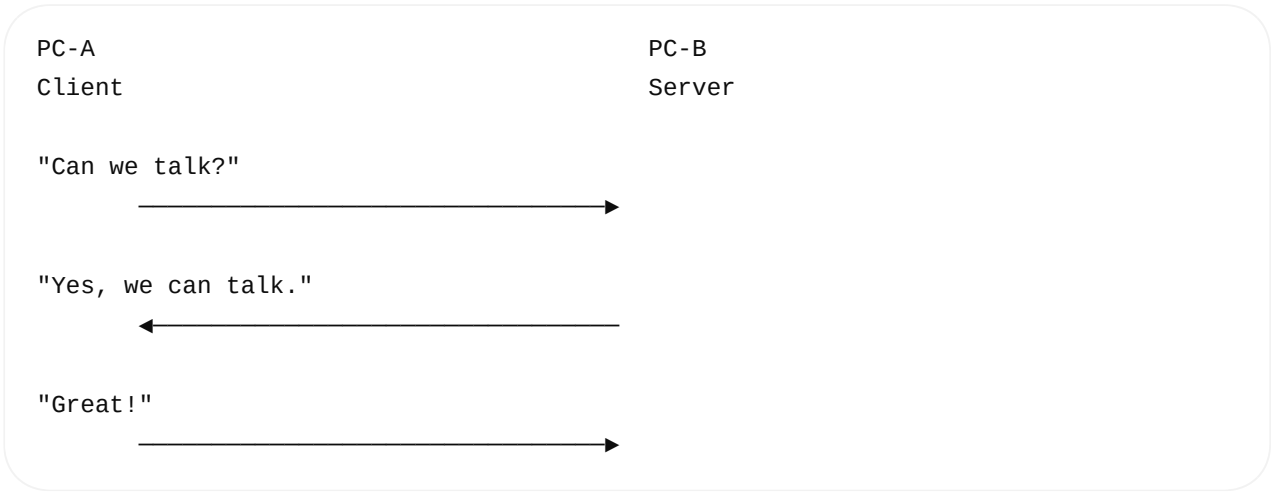
Yes. Let's make **SYN, SYN+ACK, ACK**, **flow control**, and **congestion control** very easy first. Don't worry about the complicated TCP header details yet.

1. First: What is TCP trying to do?

Imagine **PC-A wants to talk to PC-B**.



This is the basic idea of the **TCP 3-way handshake**.

The three messages are:

```
1. SYN
2. SYN + ACK
3. ACK
```

Think of it as:

> **SYN = Can we connect?**
> **SYN + ACK = Yes, I received you and I'm ready.**
> **ACK = I received your response. Let's start.**

---

2. What does SYN mean?

SYN = Synchronize

Don't worry about the word "synchronize" initially.

For interview understanding, think:

> **SYN = "I want to establish a TCP connection."**

PC-A calls:

```
connect()
```

Then the kernel TCP stack sends:

```
PC-A                                        PC-B

SYN
"I want to connect"
──────────────────────────────────────────►
```

So:

```
connect()
   ↓
TCP
   ↓
SYN
   ↓
IP Packet
   ↓
Ethernet Frame
   ↓
NIC
   ↓
PC-B
```

---

3. What does SYN + ACK mean?

PC-B receives the SYN.

PC-B says:

> "I received your request, and I also want to establish the connection."

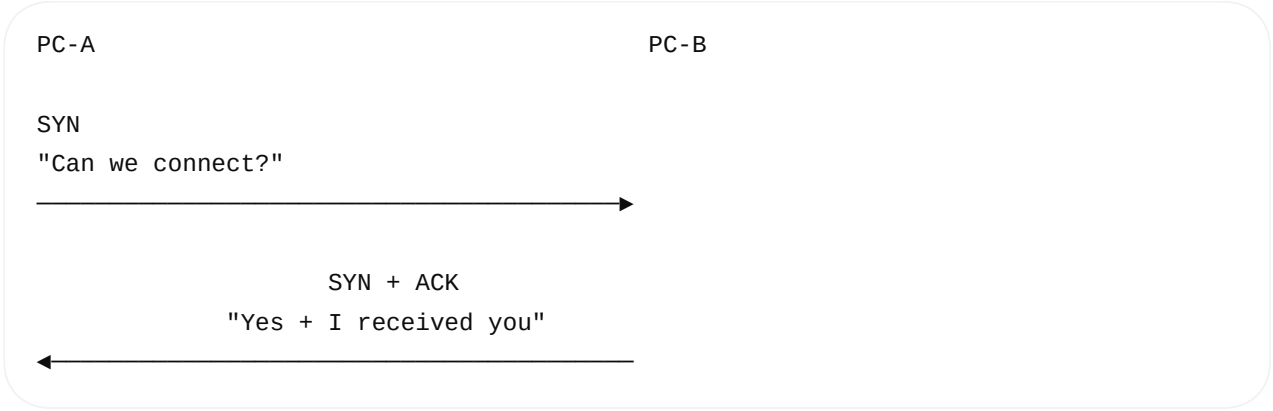
So PC-B sends:

```
SYN + ACK
```

Meaning:

```
SYN = I also want to establish the connection

ACK = I received your SYN
```

Visual:



---

4. What does ACK mean?

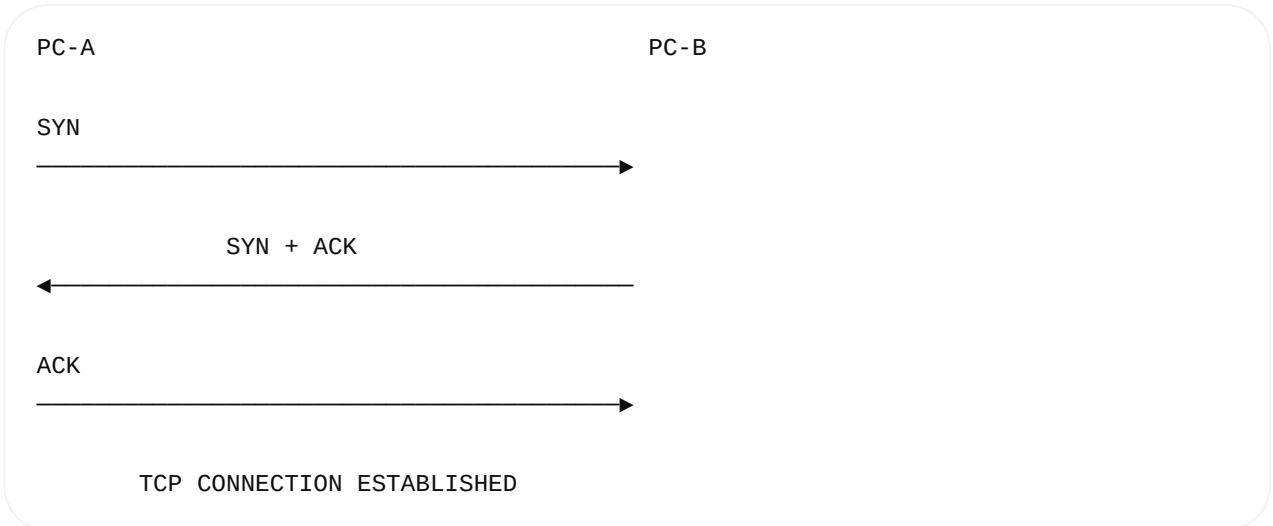
PC-A receives the `SYN + ACK`.

Now PC-A sends:

```
ACK
```

Meaning:

> **"I received your response."**



That's the whole 3-way handshake.

---

5. Easy Real-Life Example

Imagine you call your friend.

Step 1

You:

> "Hello, can you hear me?"

That's:

```
SYN
```

Step 2

Friend:

> "Yes, I can hear you. Can you hear me?"

That's:

```
SYN + ACK
```

Step 3

You:

> "Yes, I can hear you."

That's:

```
ACK
```

Now you start talking.

```
You                         Friend

"Can you hear me?"
       ──────────────────────►

"Yes, I hear you."
       ◄──────────────────────

"Great."
       ──────────────────────►

        START TALKING
```

6. Why Three Messages?

You might ask:

> Why not just two?

Because **both sides need to confirm communication**.

```
PC-A → PC-B
SYN

PC-B → PC-A
SYN + ACK

PC-A → PC-B
ACK
```

After this, both sides know:

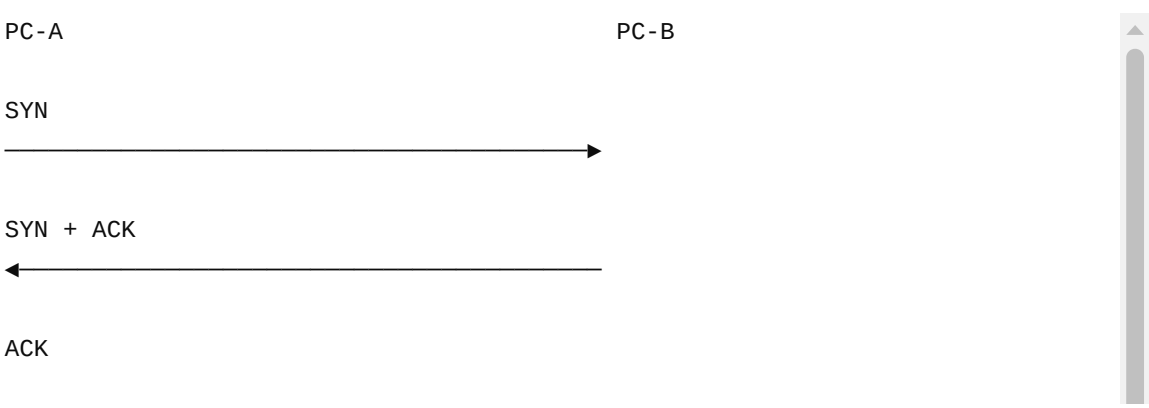
```
PC-A:
"PC-B received my request."

PC-B:
"PC-A received my response."
```

Therefore:

```
        BOTH SIDES READY
               ↓
        TCP CONNECTION
          ESTABLISHED
```

7. Now What Happens with `send()`?

After handshake:



```
──────────────────────────────────────►

       CONNECTION ESTABLISHED

HELLO
──────────────────────────────────────►

             ACK
◄──────────────────────────────────────
```

Notice something important:

SYN is not your application data.

`SYN` is a **TCP control flag** used for connection establishment.

Then:

```
send("HELLO")
```

sends actual application data.

---

8. Now Understand Flow Control

This is much easier with an example.

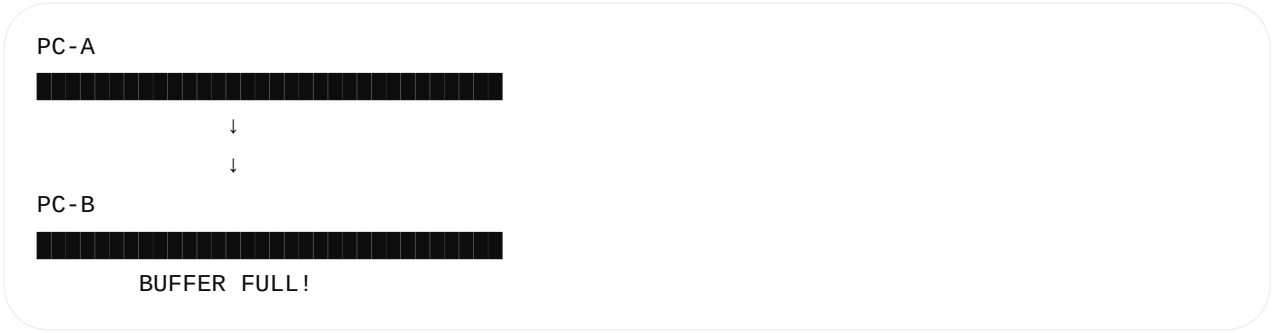
Imagine PC-A is extremely fast:

```
PC-A
████████████████████████████
              |
              | Lots of data
              ▼
           PC-B
```

But PC-B can process only a small amount at a time:

```
PC-B receive buffer


┌──────────────────────┐
| HELLO                |
| WORLD                |
| DATA                 |
|                      |
└──────────────────────┘
```

If PC-A keeps sending without limitation:



Data could be lost or the receiver could be overwhelmed.

TCP solves this using **flow control**.

---

9. Flow Control = Protect the Receiver

PC-B tells PC-A approximately:

> "I have this much buffer space available. Don't send more than I can currently handle."

For example:

```
PC-B → PC-A

Receive Window = 5000 bytes
```

Meaning conceptually:

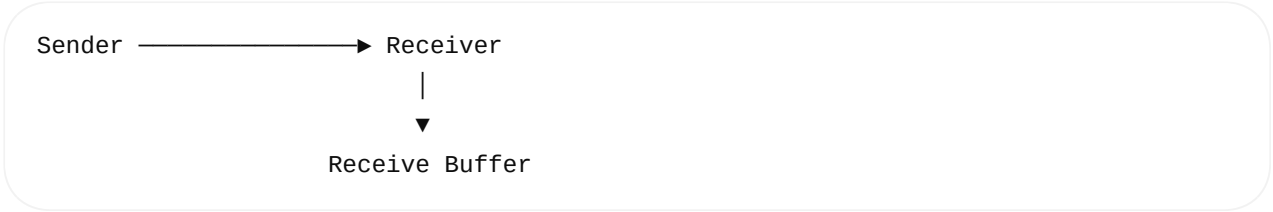
> "You can have up to about 5000 bytes of unacknowledged data outstanding based on my current receive capacity."

So:

```
PC-A                        PC-B

Send data
──────────────────►

             "My buffer has space."

Send more
──────────────────►

             "Buffer getting full."

             Receive window decreases
```

Therefore:

Flow Control

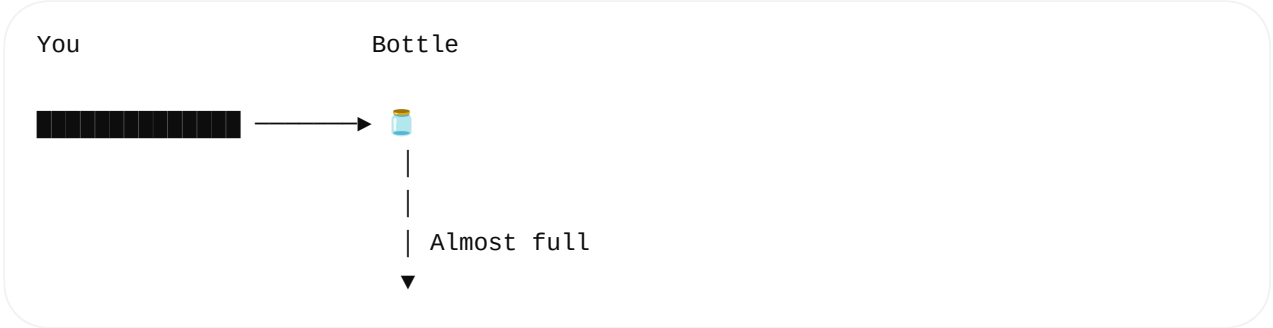


Flow control protects the receiver.

---

10. Easy Example for Flow Control

Imagine you are filling a bottle.



The bottle tells you:

> "Stop pouring so fast!"

That's similar to TCP flow control.

```
PC-A = Sender
PC-B = Receiver
PC-B's receive buffer = Bottle
```

So remember:

> **Flow control = sender should not overwhelm the receiver.**

---

11. What is Congestion Control?

Now the problem is different.

Suppose PC-B can handle lots of data.

But the **network between PC-A and PC-B is overloaded**.

```
PC-A
  |
  |
  ▼
Router
  |
  | 🚧 CONGESTED
  |
  ▼
Router
  |
  ▼
PC-B
```

Imagine 100 cars trying to enter a small road:

```
Cars
🚗 🚗 🚗 🚗 🚗 🚗 🚗 🚗
          ↓
      Small road
          ↓
       TRAFFIC JAM
```

The receiver might be perfectly fine.

The **network itself** is the problem.

TCP uses **congestion control** to reduce/increase the sending rate based on network conditions.

---

12. Flow Control vs Congestion Control

This is one of the most important interview questions.

	Flow Control	**Congestion Control**
Protects	Receiver	Network
Problem	Receiver can't process fast enough	Network is overloaded
Main concept	Receive window	Congestion window
Example	Receiver buffer full	Routers/links congested

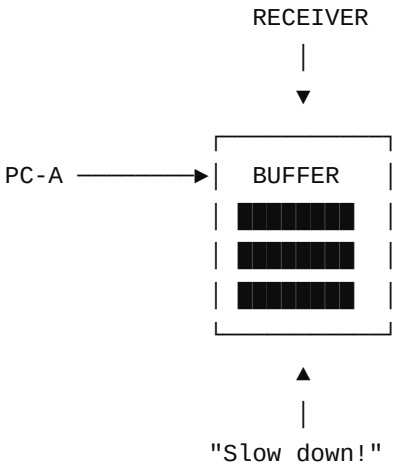
Easy memory trick:

```
FLOW CONTROL
     ↓
"Can PC-B handle this?"

CONGESTION CONTROL
     ↓
"Can the NETWORK handle this?"
```

---

13. Visual Comparison

Flow Control



The problem is:

```
          PC-B's capacity
```

Congestion Control

```
PC-A
 |
 |
 ▼
Router
 |
 |
 ▼
Router
 |
 | 🚧🚧🚧🚧
 | CONGESTION
 ▼
```

```
Router
 |
 ▼
PC-B
```

The problem is:

```
          NETWORK capacity
```

---

14. Very Important: Two Different Windows

You will hear these terms in TCP:

Receive Window (`rwnd`)

Related to:

```
FLOW CONTROL
```

Receiver tells sender how much data it can currently accept.

Congestion Window (`cwnd`)

Related to:

```
CONGESTION CONTROL
```

TCP controls how much data it should have in flight based on network congestion.

Conceptually, the amount TCP can send without waiting is constrained by both.

A simplified mental model:

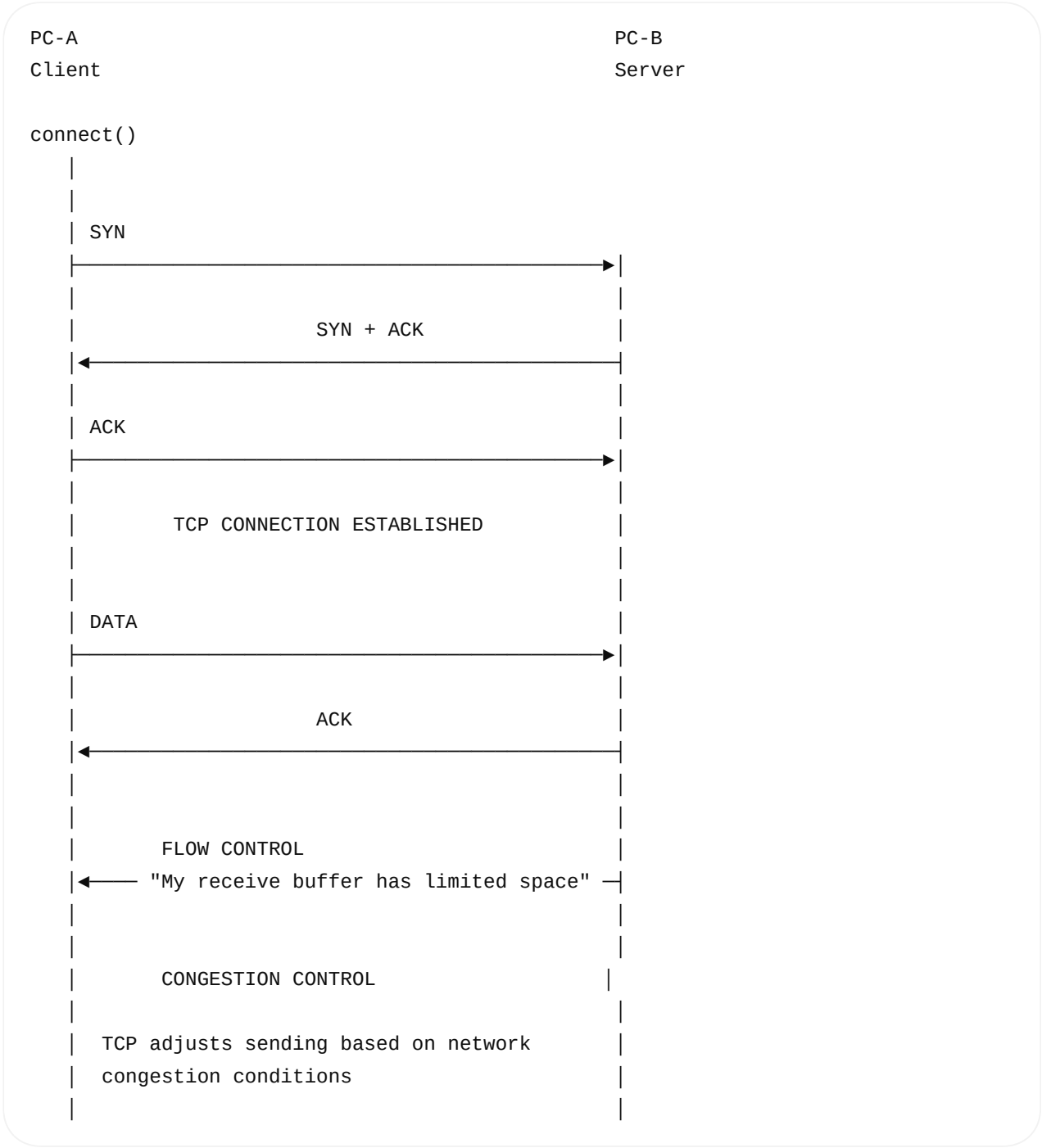
```
Effective sending window
        ≈
minimum(rwnd, cwnd)
```

You don't need to memorize the exact implementation details yet.

---

15. Put Everything Together

Now let's look at the complete TCP communication.



---

16. Where Do These Happen?

This connects to your earlier question about the Linux kernel.

SYN / ACK

Handled by:

```
Linux Kernel
      ↓
TCP Stack
```

Sequence numbers

```
Linux Kernel
      ↓
TCP
```

Retransmission

```
Linux Kernel
      ↓
TCP
```

Flow control

```
Linux Kernel
      ↓
TCP
```

Congestion control

```
Linux Kernel
      ↓
TCP
```

The application normally doesn't manually create these TCP mechanisms.

Your application does:

```
connect();
send();
recv();
```

The kernel TCP stack handles the underlying TCP protocol.

---

17. One Complete Example

Suppose:

```
PC-A
192.168.1.10
Port = 50000

        ↓

PC-B
```

```
192.168.1.20
Port = 8080
```

Application:

```
connect(sockfd, server, ...);
```

Step 1

```
PC-A → PC-B

SYN
```

> "I want to establish a connection."

Step 2

```
PC-B → PC-A

SYN + ACK
```

> "I received your request, and I am also ready."

Step 3

```
PC-A → PC-B

ACK
```

> "I received your response."

Step 4

```
send(sockfd, "HELLO", 5, 0);
```

```
PC-A → PC-B

TCP Segment
    |
```

```
    ├── TCP header
```

Step 5

PC-B acknowledges:

```
PC-B → PC-A

ACK
```

Step 6 — Flow Control

If PC-B's receive buffer becomes full:

```
PC-B

"Don't send me too much."
```

TCP reduces the amount of outstanding data according to the advertised receive window.

Step 7 — Congestion Control

If the network becomes congested:

```
PC-A → Router → Router → Router → PC-B
                  🚧
               Congestion
```

TCP adjusts its sending behavior to respond to congestion.

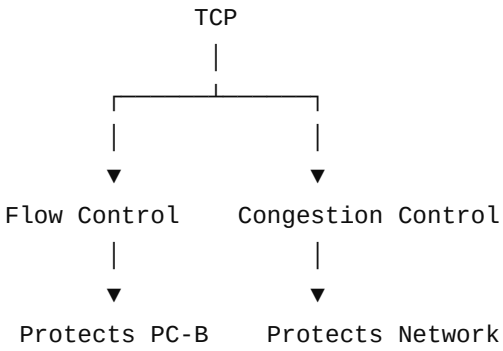
⭐ Remember These 6 Lines

For your interview, memorize the **concept**, not the complicated numbers:

```
SYN
↓
"I want to connect."

SYN + ACK
↓
"I received your request + I also want to connect."

ACK
↓
"I received your response."

Flow Control
```

```
↓
"Don't overwhelm the receiver."

Congestion Control
↓
"Don't overwhelm the network."

TCP
↓
"Make communication reliable and ordered."
```

And the most important distinction:



Next, the best thing to learn is `Sequence Number + ACK Number` with a very simple byte-by-byte example, because once that is clear, **retransmission, duplicate ACK, packet loss, and TCP reliability** become much easier.

Today 2:19 PM

> Next, the best thing to learn is Sequence Number + ACK Number with a very simple byte-by-byte example, because once that is clear, retransmission, duplicate ACK, packet loss, and TCP reliability become much easier.

expian it

Absolutely. This is the **most important TCP concept** to understand after SYN/ACK. Let's learn **Sequence Number + ACK Number** using a very small example.

1. First understand the problem

Suppose PC-A wants to send:

```
HELLO
```

There are 5 bytes:

```
H    E    L    L    O
1    2    3    4    5
```

TCP needs to answer two questions:

1. **Which bytes did I send?**
2. **Which bytes did the receiver receive?**

TCP uses:

```
Sequence Number → identifies the position of data
ACK Number      → tells what data is expected next
```

---

2. Sequence Number — "Where does my data start?"

Imagine PC-A starts TCP data with:

```
Sequence Number = 1000
```

Then:

```
H → byte 1000
E → byte 1001
L → byte 1002
L → byte 1003
O → byte 1004
```

Visual:

```
          TCP Sequence Numbers

Data:       H     E     L     L     O
            |     |     |     |     |
Seq:      1000  1001  1002  1003  1004
```

So the **sequence number identifies the position of the first byte in that TCP segment**.

For the segment containing `HELLO` :

```
Sequence Number = 1000
Data Length     = 5
```

---

3. ACK Number — "What do I want next?"

Now PC-B receives:

```
H E L L O
```

PC-B has received bytes:

```
1000
1001
1002
1003
1004
```

What byte does PC-B want next?

```
1005
```

So PC-B sends:

```
ACK = 1005
```

This is extremely important:

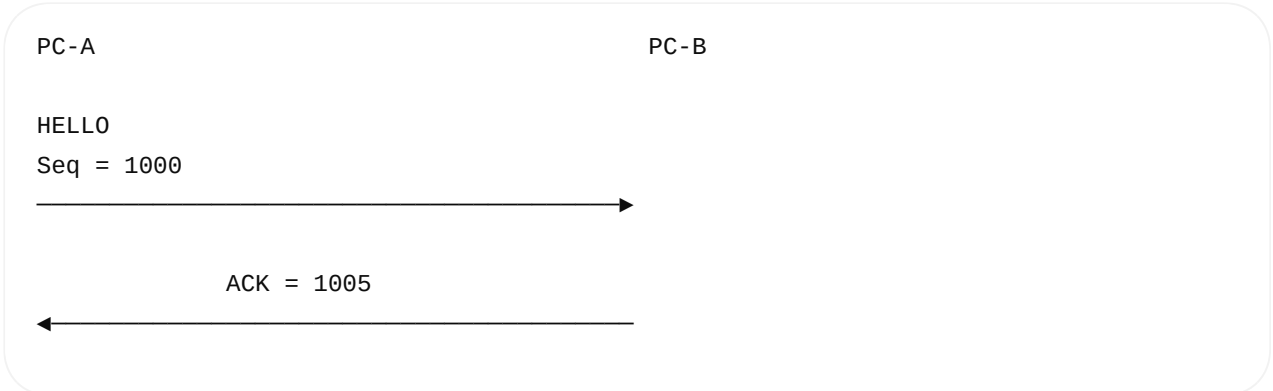
> **ACK number usually means "the next byte I expect."**

So:

```
Sequence = 1000
Data     = 5 bytes

ACK = 1000 + 5
    = 1005
```

---

4. Complete Example



Meaning:

```
PC-A:
"My data starts at byte 1000."

PC-B:
"I received bytes 1000 through 1004.
I expect byte 1005 next."
```

---

5. Why Doesn't ACK Say 1004?

This confuses many beginners.

You might think:

> "The last byte I received is 1004, so why ACK=1005?"

Because TCP ACK means:

> **"I have received everything before this number, and I want this number next."**

Therefore:

```
Received:
1000 1001 1002 1003 1004

Next expected:
1005

ACK = 1005
```

---

6. Let's Send More Data

Suppose PC-A sends:

```
ABCDEFGHIJ
```

10 bytes.

Start:

```
Sequence Number = 1000
```

Therefore:

```
A → 1000
B → 1001
C → 1002
D → 1003
E → 1004
F → 1005
G → 1006
H → 1007
I → 1008
J → 1009
```

PC-A could send this as multiple TCP segments.

For example:

```
Segment 1:

Seq = 1000
Data = ABCDE
Length = 5
```

And:

```
Segment 2:

Seq = 1005
Data = FGHIJ
Length = 5
```

---

7. PC-B Receives Both

First:

```
Segment 1
Seq = 1000
Data = ABCDE
```

PC-B sends:

```
ACK = 1005
```

Then:

```
Segment 2
Seq = 1005
```

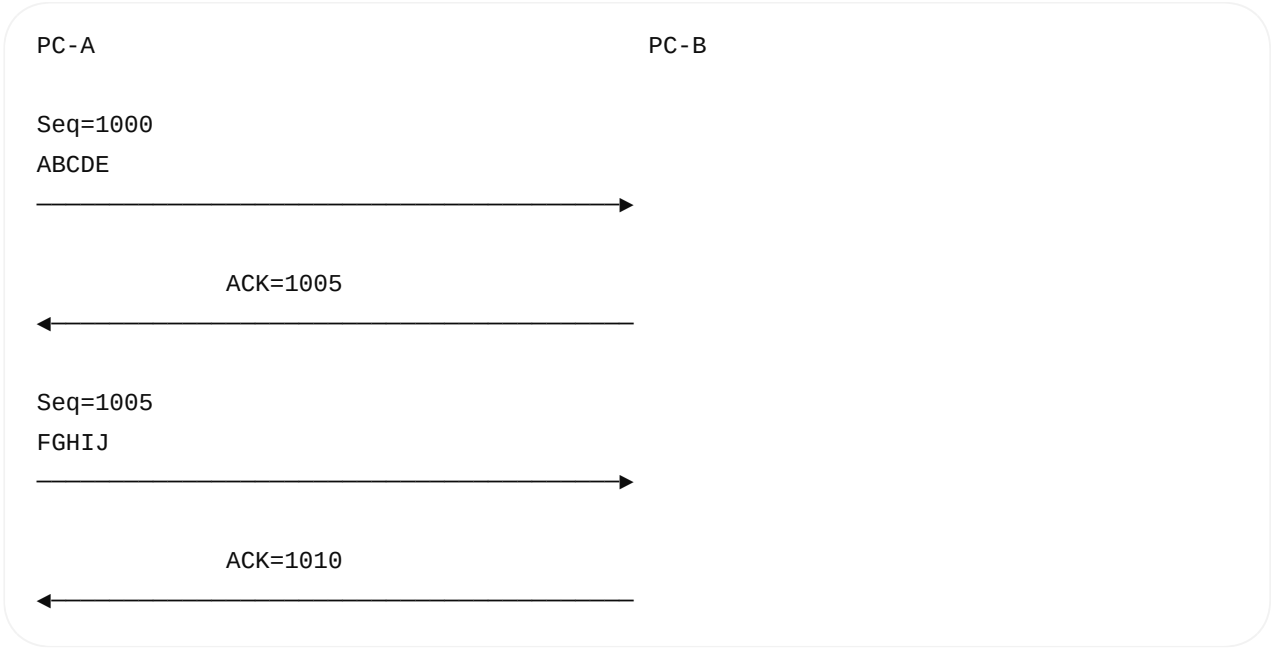
PC-B sends:

```
ACK = 1010
```

Because:

```
1000 + 10 = 1010
```

Visual:



---

8. Now the Important Part — Packet Loss

Suppose PC-A sends:

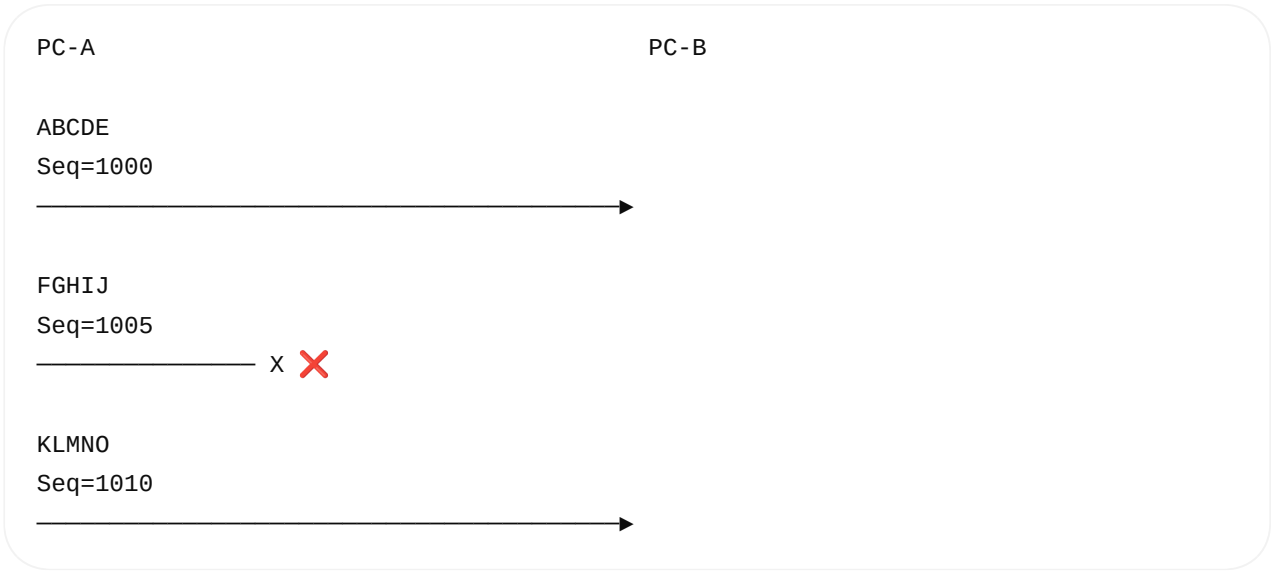
```
Segment 1 → ABCDE
Segment 2 → FGHIJ
Segment 3 → KLMNO
```

Sequence numbers:

```
Segment 1:
Seq = 1000
ABCDE

Segment 2:
Seq = 1005
FGHIJ

Segment 3:
```

```
Seq = 1010
```

But Segment 2 is lost:



What happens?

---

9. PC-B Receives Segment 3

PC-B receives:

```
KLMNO
Seq = 1010
```

But PC-B is missing:

```
FGHIJ
Seq = 1005
```

So PC-B cannot simply pretend everything is complete.

It sends an ACK indicating the next byte it still expects:

```
ACK = 1005
```

Conceptually:

> "I have everything through 1004. I still need byte 1005."

---

10. Duplicate ACK

Suppose Segment 3 arrives while Segment 2 is still missing.

PC-B may send:

```
ACK = 1005
```

Then another out-of-order segment arrives:

```
ACK = 1005
```

So PC-A may observe repeated ACKs for the same point:

```
ACK 1005
ACK 1005
ACK 1005
```

These are called:

Duplicate ACKs

They can be an indication that a segment was lost.

TCP can use duplicate ACK behavior as one signal for **fast retransmission**.

---

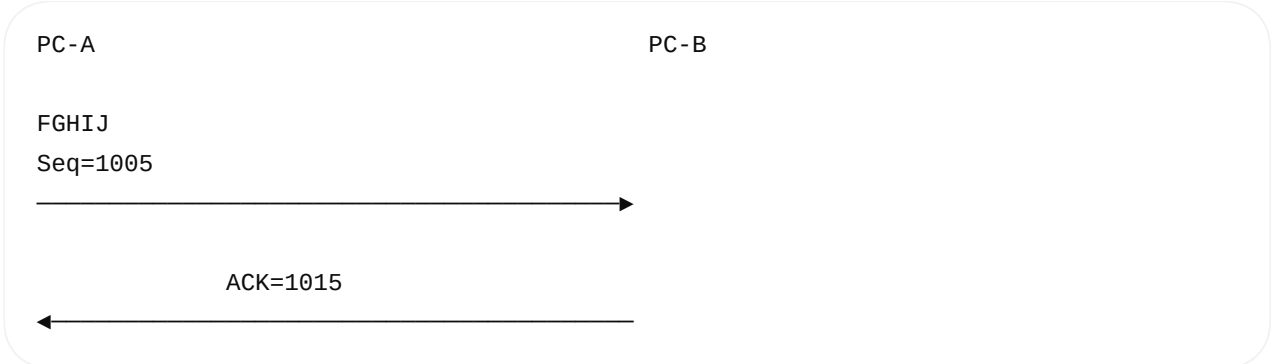
11. Retransmission

PC-A realizes:

```
Segment starting at 1005
may be missing
```

TCP retransmits:

```
FGHIJ
Seq = 1005
```



Now PC-B has:

```
ABCDE
FGHIJ
KLMNO
```

All data is in order.

---

12. Why Sequence Numbers Are So Powerful

Sequence numbers solve several problems.

Problem 1 — Ordering

Network:

```
Segment 1
Segment 3
Segment 2
```

TCP can use sequence numbers to reconstruct:

```
Segment 1
Segment 2
Segment 3
```

Problem 2 — Detecting Missing Data

Expected:

```
1000
1005
1010
```

Received:

```
1000
1010
```

TCP knows:

```
1005 is missing
```

Problem 3 — Duplicate Data

Suppose the same segment arrives twice:

```
Seq=1005
FGHIJ

Seq=1005
FGHIJ
```

TCP can recognize the duplicate using the sequence number.

Problem 4 — Acknowledgement

TCP knows exactly how much data has been received.

---

13. One Important Correction to Your Mental Model

Don't think:

```
❌ One packet = one sequence number
```

Think:

```
✅ Sequence number identifies the first byte
   of data carried by a TCP segment.
```

Example:

```
Segment:

Seq = 1000
Data = ABCDE
```

means:

```
A = 1000
B = 1001
C = 1002
D = 1003
E = 1004
```

Therefore the next expected byte is:

```
ACK = 1005
```

---

14. What About SYN?

There's one more important detail.

TCP sequence numbers are also involved in the handshake.

Suppose:

```
PC-A → PC-B

SYN
Seq = 1000
```

SYN consumes one sequence-number position.

PC-B responds:

```
SYN + ACK
Seq = 5000
ACK = 1001
```

Why:

```
1000 + 1 = 1001
```

Then PC-A:

```
ACK = 5001
```

because PC-B's SYN started at:

```
5000
```

and SYN consumes one sequence number.

So:

```
PC-A                                    PC-B

SYN
Seq=1000
──────────────────────────────────────────►

              SYN + ACK
              Seq=5000
              ACK=1001
◄──────────────────────────────────────────
```

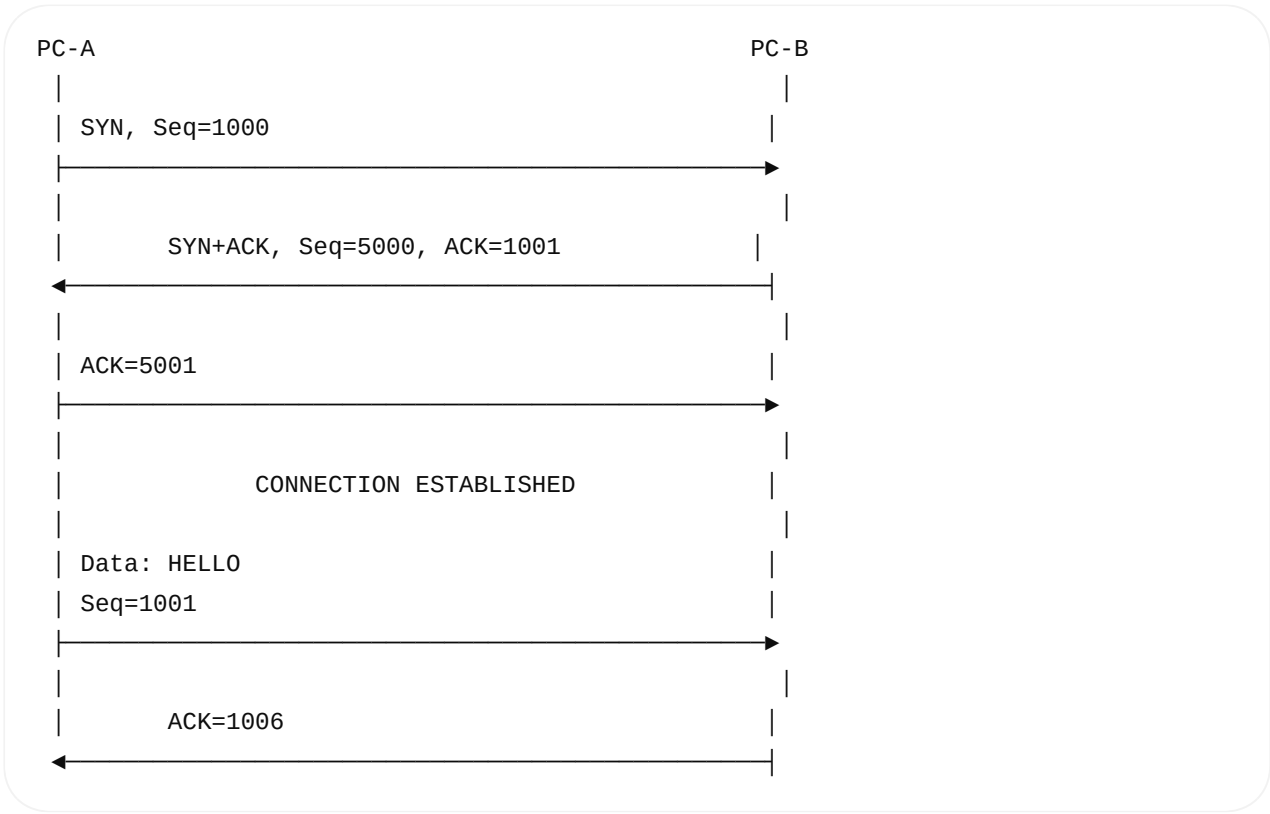
```
ACK=5001
──────────────────────────────────────────────►
```

Now application data can start.

---

15. Put Handshake + Data Together

This is the complete picture:



Why is the data starting at `1001` ?

Because:

```
SYN started at 1000
SYN consumes one sequence number

1000 + 1 = 1001
```

And:

```
HELLO = 5 bytes

1001 + 5 = 1006

ACK = 1006
```

---

16. The Best Way to Remember

Use this simple formula:

Sequence Number

```
SEQ = "Where does my data start?"
```

ACK Number

```
ACK = "What byte do I want next?"
```

Example:

```
Sender:

SEQ = 1000
DATA = 5 bytes
```

Receiver:

```
ACK = 1005
```

Because:

```
1000 + 5 = 1005
```

---

17. Visual Memory Trick

Imagine a numbered road:

```
1000  1001  1002  1003  1004  1005  1006
  H     E     L     L     O     ?     ?
  └──────────── received ─────────┘
                                  ↑
                            ACK = 1005
```

ACK points to the **next missing/expected byte** in the normal in-order case.

If byte 1005 is missing:

```
1000  1001  1002  1003  1004  1005  1006
  H     E     L     L     O     X     G
  └──────────── received ─────────┘     received
```

```
                ↑
ACK = 1005
```

Even though data after the gap may have arrived, the cumulative ACK still indicates:

> **"I need 1005 next."**

⭐ Interview Answer

If the interviewer asks:

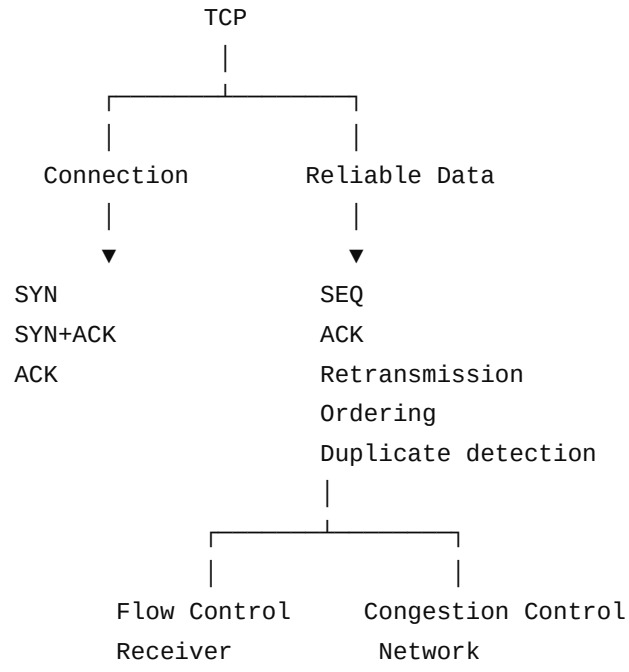
"What is the difference between TCP sequence number and acknowledgement number?"

Say:

> **The TCP sequence number identifies the position of the first data byte carried in a TCP segment. The acknowledgement number indicates the next byte the receiver expects, so it effectively acknowledges all bytes before that number. For example, if a segment starts at sequence number 1000 and carries 500 bytes, the receiver can send ACK 1500, meaning it has received through byte 1499 and expects byte 1500 next.**

That's a strong interview answer.

Your TCP mental model now



The **next concept after this** is **MSS, MTU, TCP segmentation, IP fragmentation, and how a large `send()` becomes multiple TCP segments → IP packets → Ethernet**

frames. That will connect directly with the Layer 2/3 learning you did earlier.

go head with obrivation

Sure. Let's continue **Day 4** with the abbreviations first, then connect them to **MSS → MTU → TCP Segment → IP Packet → Ethernet Frame**.

Day 4 — Part 2: TCP Abbreviations + Segmentation

1. Important TCP abbreviations

Abbreviation	Full form	Easy meaning
TCP	Transmission Control Protocol	Reliable transport protocol
UDP	User Datagram Protocol	Connectionless transport protocol
SYN	Synchronize	Start TCP connection
ACK	Acknowledgment	Confirms received data
FIN	Finish	Gracefully close connection
RST	Reset	Immediately terminate/reset connection
PSH	Push	Ask TCP to push data toward application
URG	Urgent	Indicates urgent data
SEQ	Sequence Number	Position of TCP data
MSS	Maximum Segment Size	Maximum TCP payload in one segment
MTU	Maximum Transmission Unit	Maximum IP packet size a link normally carries
RTT	Round-Trip Time	Time for data/request and response
RTO	Retransmission Timeout	Timeout used for retransmission
ACK	Acknowledgment	Next byte expected

Abbreviation	Full form	Easy meaning
rwnd	Receive Window	Receiver's available TCP receive capacity
cwnd	Congestion Window	Sender's congestion-control limit
IP	Internet Protocol	Layer-3 addressing/routing
NIC	Network Interface Controller/Card	Network hardware
ARP	Address Resolution Protocol	IPv4 → MAC mapping
FCS	Frame Check Sequence	Ethernet error-detection field

---

2. The 4 Terms You MUST Understand

The most important new terms today are:

```
MTU
 ↓
MSS
 ↓
TCP Segment
 ↓
IP Packet
```

Let's understand them with one simple example.

---

3. What is MTU?

MTU = Maximum Transmission Unit

It means:

> **The maximum size of an IP packet that a network interface/link normally carries without IP fragmentation.**

A common Ethernet MTU is:

```
1500 bytes
```

So conceptually:

```
Ethernet network

Maximum IP packet
        ↓
    1500 bytes
        ↓
       MTU
```

---

4. What is MSS?

MSS = Maximum Segment Size

MSS tells TCP:

> **How much application data can fit inside one TCP segment.**

For a typical Ethernet path:

```
MTU = 1500 bytes
```

IPv4 header:

```
20 bytes
```

TCP header:

```
20 bytes
```

Therefore, typical MSS:

```
1500 - 20 - 20
      ↓
    1460
```

So:

```
MSS = 1460 bytes
```

This is the common example you should remember.

---

5. Visualize MTU and MSS

Think of an envelope.

```
        IP Packet
┌──────────────────────────────────┐
│ IP Header       20 bytes         │
├──────────────────────────────────┤
│ TCP Header      20 bytes         │
├──────────────────────────────────┤
│                                  │
│ TCP Payload                      │
│                                  │
│ Maximum ≈ 1460 bytes             │
│                                  │
└──────────────────────────────────┘

Total = 1500 bytes
       ↑
      MTU
```

Therefore:

```
MTU = complete IP packet size

MSS = TCP data/payload size
```

---

6. Very Important Difference

Don't confuse:

```
MTU ≠ MSS
```

Remember:

```
MTU
↓
IP packet maximum size

MSS
↓
TCP payload maximum size
```

Typical Ethernet + IPv4 example:

```
MTU = 1500
IP header = 20
TCP header = 20

MSS = 1460
```

7. What Happens If Application Sends 5000 Bytes?

Suppose:

```
send(sockfd, buffer, 5000, 0);
```

Don't think:

```
❌ 5000-byte TCP segment
```

TCP can divide the byte stream into multiple segments.

With an example MSS of 1460:

```
5000 bytes
    |
    ├──────────────┐
    |              |
    ▼              ▼
 Segment 1       1460
 Segment 2       1460
 Segment 3       1460
 Segment 4        620
```

Check:

```
1460 + 1460 + 1460 + 620
= 5000
```

---

8. Complete Encapsulation

Now connect everything from your previous lessons.

Application sends:

```
5000 bytes
```

TCP:

```
5000 bytes
     ↓
TCP segmentation
     ↓
```

```
1460
1460
1460
620
```

Then TCP adds its header.

For Segment 1:

```
TCP Segment
┌──────────────────────┐
| TCP Header           |
| 20 bytes             |
├──────────────────────┤
| Data                 |
| 1460 bytes           |
└──────────────────────┘
```

Total:

```
20 + 1460 = 1480 bytes
```

---

9. Then IP Adds Its Header

```
IP Packet
┌──────────────────────────────────┐
| IP Header       20 bytes         |
├──────────────────────────────────┤
| TCP Header      20 bytes         |
├──────────────────────────────────┤
| TCP Data        1460 bytes       |
└──────────────────────────────────┘

Total = 1500 bytes
```

Perfectly fits:

```
MTU = 1500
```

---

10. Then Ethernet

Ethernet adds Layer-2 framing around the IP packet.

Conceptually:

```
Ethernet Frame
┌──────────────────────────────────┐
| Ethernet Header                  |
├──────────────────────────────────┤
| IP Header                        |
├──────────────────────────────────┤
| TCP Header                       |
├──────────────────────────────────┤
| Application Data                 |
├──────────────────────────────────┤
| FCS                              |
└──────────────────────────────────┘
```

So the complete process is:

```
Application
    |
    | 5000 bytes
    ▼
TCP
    |
    | segmentation
    ▼
TCP Segments
    |
    ▼
IP Packets
    |
    ▼
Ethernet Frames
    |
    ▼
NIC
    |
    ▼
Network
```

---

11. Very Important Correction

Earlier you were thinking:

> "Each segment is sent to Layer 1."

The better way to think about it is:

```
TCP Segment
    ↓
IP Packet
    ↓
Ethernet Frame
    ↓
Physical transmission
```

The **actual thing transmitted over an Ethernet link is the Ethernet frame**, represented as electrical/optical/radio signals at the physical layer.

So:

```
TCP → creates segment
IP → encapsulates segment into packet
Ethernet → encapsulates packet into frame
NIC/PHY → transmits bits/signals
```

---

12. Sequence Numbers with Segmentation

Now combine today's previous topic.

Suppose:

```
Application = 5000 bytes
MSS = 1460
Initial TCP sequence = 1000
```

TCP can create:

```
Segment 1
Seq = 1000
Data = 1460 bytes
```

Next:

```
Segment 2
Seq = 2460
Data = 1460 bytes
```

Because:

```
1000 + 1460 = 2460
```

Next:

```
Segment 3
Seq = 3920
Data = 1460 bytes
```

Next:

```
Segment 4
Seq = 5380
Data = 620 bytes
```

Because:

```
3920 + 1460 = 5380
```

---

13. Receiver ACK

Suppose everything arrives correctly.

After Segment 1:

```
Seq = 1000
Length = 1460

ACK = 2460
```

After Segment 2:

```
Seq = 2460
Length = 1460

ACK = 3920
```

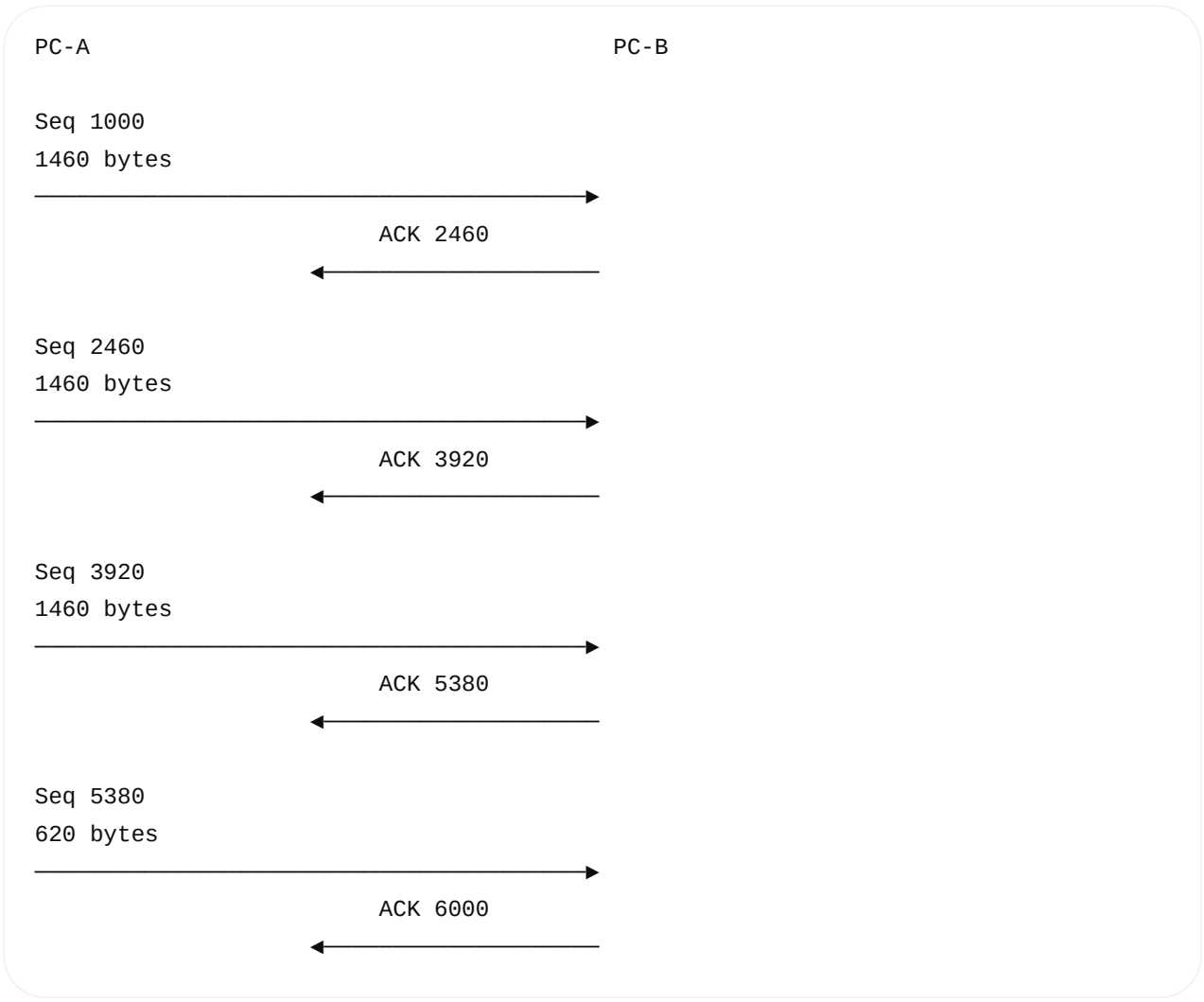
After Segment 3:

```
Seq = 3920
Length = 1460

ACK = 5380
```

After Segment 4:

```
Seq = 5380
Length = 620

ACK = 6000
```

Visual:



Now you can see why **sequence numbers + MSS + ACK numbers** are connected.

---

14. Where Does MSS Come From?

During the TCP handshake, TCP endpoints can advertise their MSS using a **TCP option**.

Conceptually:

MSS is therefore negotiated/advertised during connection establishment.

Important:

> MSS is about TCP payload size, while MTU is about the maximum IP packet size on a link/path.

---

15. What If MTU Is Smaller?

Suppose the path has:

```
MTU = 1200
```

Then a sender cannot simply assume:

```
MSS = 1460
```

for that path.

For IPv4 with 20-byte IP and 20-byte TCP headers, a corresponding payload size would be approximately:

```
1200 - 20 - 20
= 1160
```

This is why MSS can depend on the path and TCP options.

---

16. TCP Options

You will see terms such as:

MSS

Maximum Segment Size.

Window Scaling

Allows larger effective TCP windows than the original 16-bit window field alone.

SACK

Selective Acknowledgment

Allows a receiver to tell the sender about specific blocks of data that have arrived, which can make loss recovery more efficient.

Timestamps

TCP timestamps can assist with RTT measurement and other TCP mechanisms.

For now, remember:

```
TCP Options
├── MSS
├── Window Scale
├── SACK
└── Timestamps
```

---

17. MTU vs MSS — Interview Question

Interviewer:

"What is the difference between MTU and MSS?"

Good answer:

> **MTU is the maximum size of an IP packet that can normally be transmitted over a link without fragmentation. MSS is the maximum amount of TCP application payload carried in one TCP segment. For a typical 1500-byte Ethernet MTU with 20-byte IPv4 and 20-byte TCP headers, the common MSS is 1460 bytes.**

---

18. Another Important Abbreviation: RTT

RTT = Round-Trip Time

Suppose PC-A sends:

```
TCP Segment
──────────►
```

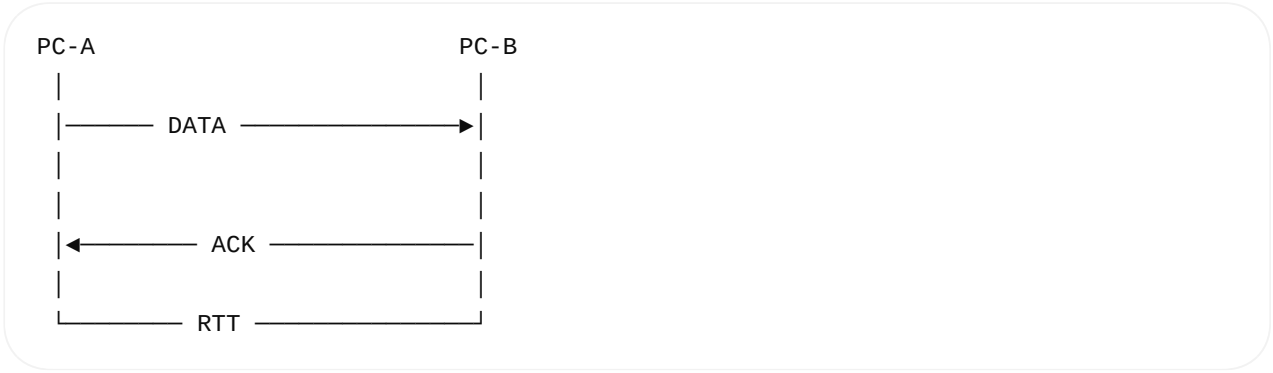
and receives the ACK:

```
ACK
◄──────────
```

The elapsed time is approximately:

```
RTT
```

Visual:



TCP uses RTT measurements as part of its retransmission timing calculations.

---

19. RTO

RTO = Retransmission Timeout

Conceptually:

```
Send segment
     |
     ▼
Wait for ACK
     |
     ├── ACK arrives
     |       ↓
     |     Good
     |
     └── Timeout
             ↓
       Retransmission
```

Don't think:

```
RTO = always exactly 1 second
```

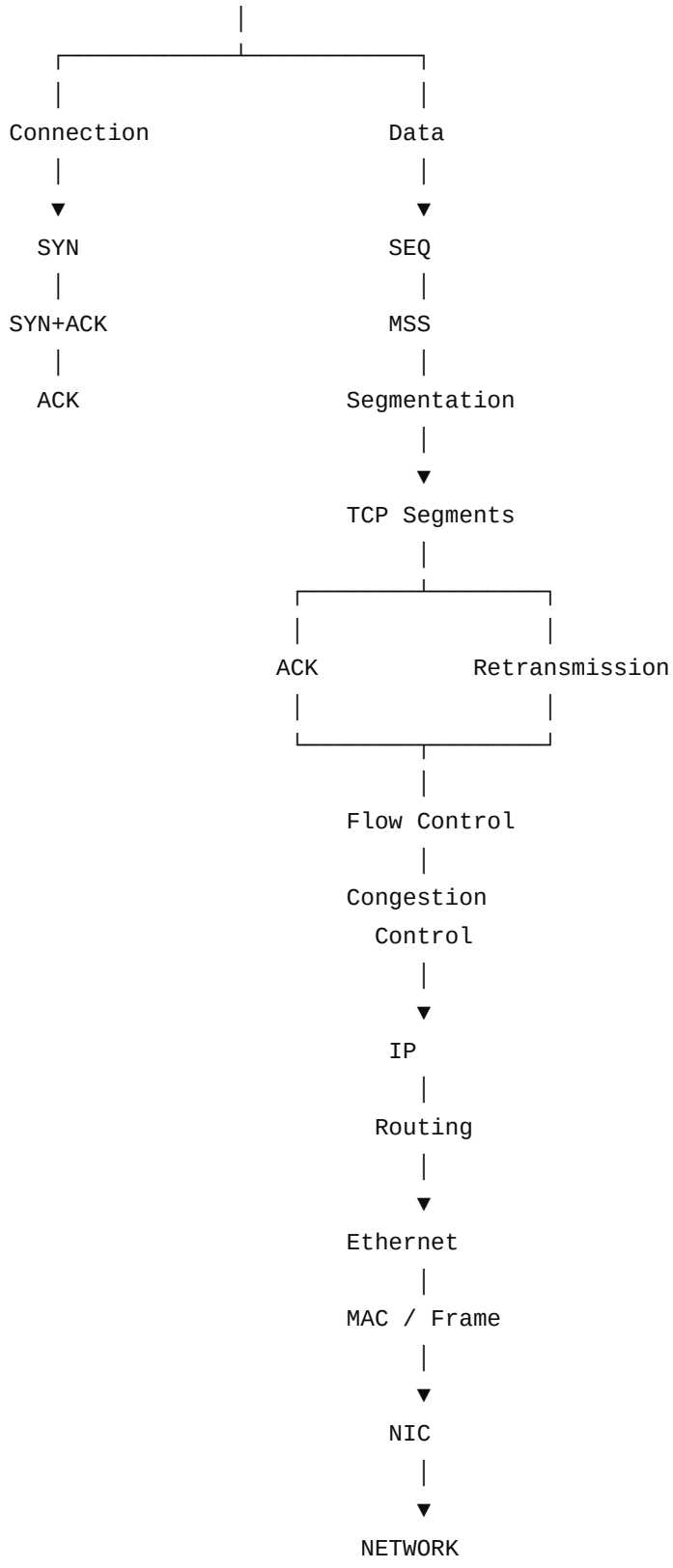
The actual TCP RTO is dynamically calculated based on measured network behavior.

---

20. Day 4 Complete Mental Model

Now you have almost the complete TCP picture:

```
                    APPLICATION
                         |
                      send()
                         |
                         ▼
                    TCP SOCKET
```

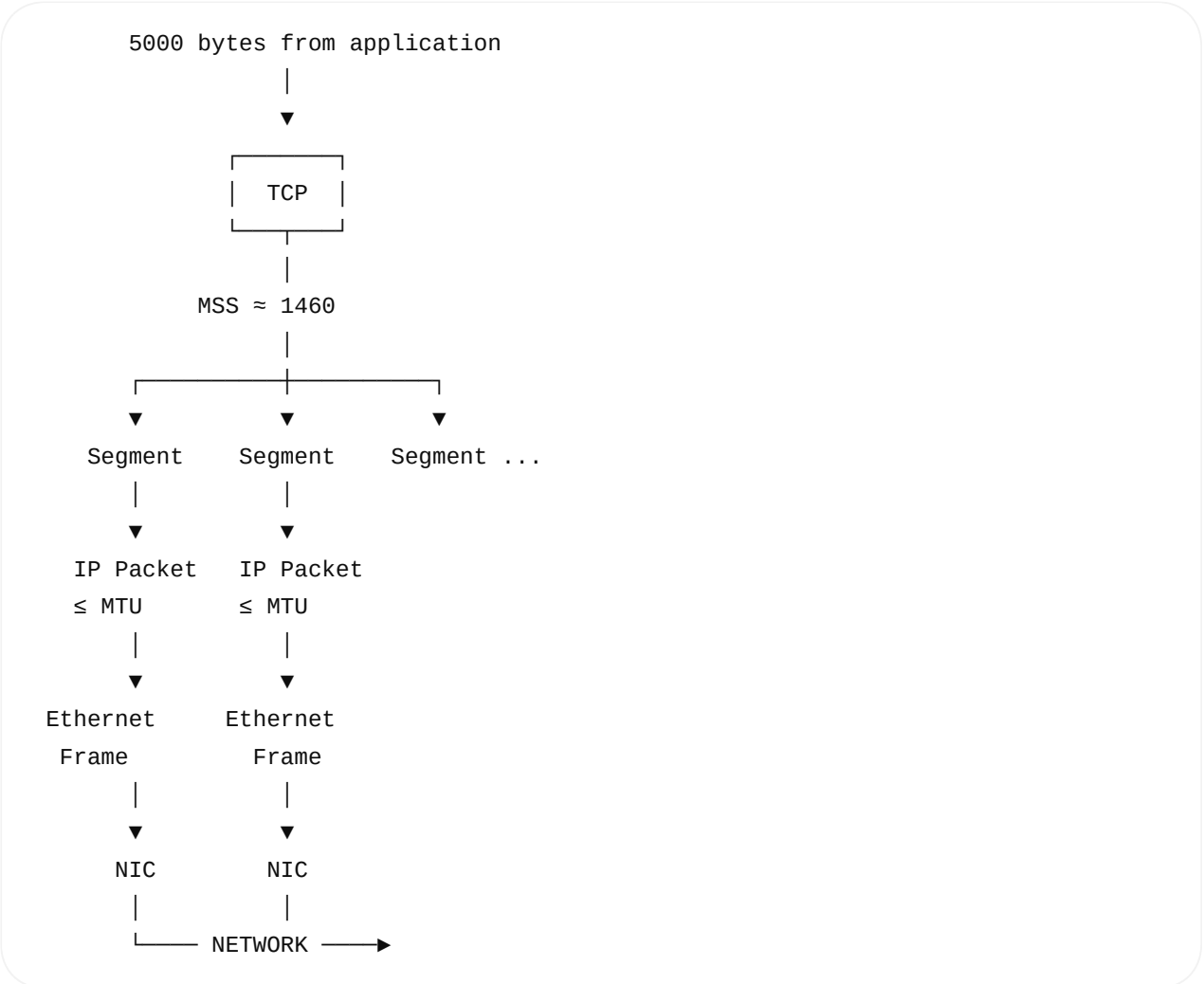


⭐ Day 4 Abbreviations You Should Memorize

```
TCP   = Transmission Control Protocol
UDP   = User Datagram Protocol

SYN   = Synchronize
ACK   = Acknowledgment
FIN   = Finish
RST   = Reset
```

```
SEQ   = Sequence Number
MSS   = Maximum Segment Size
MTU   = Maximum Transmission Unit

RTT   = Round-Trip Time
RTO   = Retransmission Timeout

rwnd  = Receive Window
cwnd  = Congestion Window

ARP   = Address Resolution Protocol
IP    = Internet Protocol
NIC   = Network Interface Controller/Card
FCS   = Frame Check Sequence
```

🔑 One final picture to remember



The next Day 4 topic to master is TCP connection states (`LISTEN` **,** `SYN-SENT` **,** `SYN-RECEIVED` **,** `ESTABLISHED` **,** `FIN-WAIT` **,** `TIME-WAIT` **) and how they relate directly to** `socket()` **,** `bind()` **,** `listen()` **,** `connect()` **,** `accept()` **,and** `close()` **.**